
FairLangProc Documentation

Release 0.1.9

Arturo Perez-Peralta

May 13, 2026

CONTENTS

1	Getting Started	3
1.1	Why FairLangProc?	3
1.2	Overview	4
1.3	Installation	4
1.4	Quick Start	5
1.5	Further Reading	5
2	Datasets	7
2.1	Overview	7
2.2	API Reference and usage examples	7
3	Metrics	9
3.1	Supported Metrics	9
3.2	Embedding Metrics	9
3.3	Probability Metrics	11
3.4	Generated Text Metrics	14
4	Algorithms	17
4.1	Supported processors	17
5	Tutorials	27
5.1	Datasets	27
5.2	Metrics	27
5.3	Bias Mitigation	27
5.4	End-to-End Workflow	28
5.5	Notebook Index	28
5.6	Requirements	28
6	Citation and bibliography	29
6.1	Citation	29
6.2	Related Publications	29

The **Fair Language Processing** (FairLangProc) package is an extensible open-source Python library containing techniques developed by the research community to help detect and mitigate bias in Natural Language Processing throughout the AI application lifecycle.

The FairLangProc package includes:

1. **Datasets** - Benchmarks to test for biases in NLP models
2. **Metrics** - Fairness measures based on different philosophies to quantify bias
3. **Algorithms** - Bias mitigation techniques at pre-processing, in-processing, and intra-processing stages

It has been created with the intention of encouraging the use of bias mitigation strategies in the NLP community, with the hope of democratizing these tools for the ever-increasing set of NLP practitioners.

Tip

The companion paper provides a comprehensive introduction to the concepts and capabilities, with all code available in the [notebooks](#).

Note

If you use this library in your research, please cite our [arXiv paper](#).

GETTING STARTED

This chapter provides an introduction to FairLangProc, including installation, quick start examples, and a comparison with other fairness tools.

1.1 Why FairLangProc?

A unified framework for fairness in NLP with comprehensive coverage of datasets, metrics, and algorithms.

There are several tools available for bias mitigation in the fair ML landscape:

- **AI Fairness 360 (aif360)** - The main inspiration for this project. Provides bias mitigation methods for tabular data but lacks NLP-specific features.
- **Fairlearn** - Offers fairness tools but is not focused on NLP.
- **AllenNLP** - Provides embedding metrics and debiasing methods but development has been discontinued.
- **Dbias** - Offers a pipeline for bias evaluation but uses a custom methodology.
- **LangFair** - Allows generating custom datasets for fairness evaluation but lacks debiasing algorithms.
- **langtest** - Comprehensive evaluation library but with limited bias mitigation capabilities.

FairLangProc addresses these limitations by providing:

- **NLP-First Design** - Specifically designed for text data with HuggingFace transformers integration
- **Comprehensive Coverage** - 13+ benchmark datasets, 8+ fairness metrics, and 9+ debiasing algorithms
- **Pre/In/Intra-Processing** - Support for all stages of the ML pipeline
- **Unified Interface** - Consistent API across all modules
- **Active Development**

1.2 Overview

Table 1: FairLangProc Main Functionalities

DATASETS	METRICS	ALGORITHMS
BBQ	WEAT	Pre-processors:
BEC-Pro	LPBS	• CDA
BOLD	CBS	• Projection
BUG	CPS	• BLIND
CrowS-Pairs	AUL	In-processors:
GAP	HONEST	• ADELE
HolisticBias	DR	• EAR
HONEST	SA	• Regularizers
StereoSet		Intra-processors:
UnQover		• EAT
WinoBias		• MoDDiffy
WinoBias+		
WinoGender		

1.3 Installation

1.3.1 Prerequisites

FairLangProc requires:

- Python ≥ 3.10
- numpy $\geq 2.2.4$
- pandas $\geq 2.2.3$
- scikit-learn $\geq 1.6.1$
- torch $\geq 2.6.0$
- transformers $\geq 4.47.1$
- datasets $\geq 3.4.1$
- adapter-transformers $\geq 1.1.0$
- pytest $\geq 8.4.1$

1.3.2 Install from PyPI

```
pip install FairLangProc
```

1.3.3 Install from source

```
git clone https://github.com/arturo-perez-peralta/FairLangProc
cd FairLangProc
pip install -e .
```

1.4 Quick Start

Load a dataset for bias evaluation:

```
from FairLangProc.datasets import BiasDataLoader

ageBBQ = BiasDataLoader(dataset = 'BBQ', config = 'Age')
print(ageBBQ['data'][0])
```

Measure bias using common metrics:

```
from FairLangProc.metrics import CBS

model = AutoModelForMaskedLM.from_pretrained('bert-base-uncased')
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
target_words = [("John", "Mamadouk", "Liu"), ("white", "black", "asian"), ("white",
↪ "black", "asian")]
sentences = ["[MASK] is a [MASK]", "The [MASK] kid got [MASK] results", "The [MASK] kid_
↪ wanted to be a [MASK]"]
fill_words = ["engineer", "outstanding", "doctor"]
mask_indices = [0, 1, 1]

CBSscore = CBS(
    model = model,
    tokenizer = tokenizer,
    sentences = sentences,
    target_words = target_words,
    fill_words = fill_words,
    mask_indices = mask_indices
)
```

Mitigate bias with an algorithm:

```
from transformers import BertForSequenceClassification
from FairLangProc.algorithms.inprocessors import EARModel

model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

EARRegularizer = EARModel(
    model = model,
    ear_reg_strength = 0.01
)
```

1.5 Further Reading

For a more in-depth reading of the methodology and implementation consult:

- [GitHub Repository](#) - Source code and issues
- [arXiv Paper](#) - Academic publication

DATASETS

The FairLangProc datasets module provides access to standard benchmarks for evaluating gender, racial, religious, and other social biases in NLP models.

2.1 Overview

The `BiasDataLoader` is the main entry point for loading bias evaluation datasets. It supports multiple output formats and dataset configurations.

Table 1: Supported Benchmark Datasets

Data Set	Size	Bias target	Reference
BBQ	58,492	Gender, race, religion,...	(Parrish et al., 2021)
BEC-Pro	5,400	Gender	(Bartl et al., 2020)
BOLD	23,679	Gender, race, religion,...	(Dhamala et al., 2021)
BUG	108,419	Gender	(Levy et al., 2021)
Crow-SPairs	1,508	Age, disability, gender, nationality,...	(Nangia et al., 2020)
GAP	8,908	Gender	(Webster et al., 2018)
HolisticBias	460,000	Age, disability, gender, nationality,...	(Smith et al., 2022)
HONEST	420	Gender	(Nozza et al., 2021)
StereoSet	16,995	Gender, race, religion,...	(Nadeem et al., 2020)
UnQover	30	Gender, nationality, race,...	(Li et al., 2020)
WinoBias+	1,367	Gender	(Vanmassenhove et al., 2021)
WinoBias	3,160	Gender	(Zhao et al., 2018)
WinoGender	720	Gender	(Rudinger et al., 2018)

2.2 API Reference and usage examples

`FairLangProc.datasets.fairness_datasets.BiasDataLoader`(*dataset*: *str* | *None* = *None*, *config*: *str* | *None* = *None*, *format*: *str* = 'hf', *benchmark_path*: *str* | *None* = *None*) → Dict[*str*, pandas.DataFrame | List[*str*] | Dataset | datasets.Dataset] | *None*

Load specified bias evaluation dataset.

Requires downloading the Fair-LLM-Benchmark repository (<https://github.com/i-gallegos/Fair-LLM-Benchmark>), credits to Isabel O. Gallegos et al).

Parameters

- **dataset** (*str*) – name of the dataset.

- **config** (*str*) – dataset configuration if applicable.
- **format** (*str*) – output format - 'raw', 'hf' (hugging face), or 'pt' (pytorch).
- **benchmark_path** (*str*) – path where the Fair-LLM-Benchmark resides. If none, it looks for it in FairLangProc/FairLangProc/datasets/Fair-LLM-Benchmark

Returns

dataDict – Dictionary with datasets in the appropriate format.

Return type

dict

Example

```
>>> from FairLangProc.datasets import BiasDataLoader
>>> BiasDataLoader()
Available datasets:
=====
BBQ
BEC-Pro
BOLD
BUG
CrowS-Pairs
GAP
HolisticBias
StereoSet
WinoBias+
WinoBias
Winogender
>>> BiasDataLoader(dataset = 'BBQ')
Available configurations:
=====
Age
Disability_Status
Gender_identity
Nationality
Physical_appearance
Race_ethnicity
Race_x_gender
Race_x_SES
Religion
SES
Sexual_orientation
all
>>> ageBBQ = BiasDataLoader(dataset = 'BBQ', config = 'Age')
```

See also

- *Tutorials* - Interactive Jupyter notebooks (*DemoDatasets.ipynb*) <<https://github.com/arturo-perez-peralta/FairLangProc/blob/main/notebooks/DemoDatasets.ipynb>> demonstrating dataset usage

METRICS

FairLangProc provides comprehensive fairness metrics to measure discrimination in NLP models.

3.1 Supported Metrics

FairLangProc supports different fairness metrics to measure discrimination in NLP. Broadly, they can be classified into three categories:

- **Embedding metrics** (WEAT, SEAT): if they measure bias by examining the model's hidden representations of input text.
- **Probability metrics** (LPBS, CBS, CPS, AUL): if they measure bias by computing the probabilities of certain tokens or sentences.
- **Generated text metrics** (DR, SA, HONEST): if they measure bias by examining text generated by the model, looking for harmful or stereotypical words.

The implemented metrics are:

- Generalized association tests (WEAT) (Caliskan et al., 2016).
- *Log Probability Bias Score (LPBS)* (Kurita et al., 2019).
- *Categorical Bias Score (CBS)* (Ahn et al., 2021).
- *CrowS-Pairs Score (CPS)* (Nangia et al., 2020).
- *All Unmasked Score (AUL)* (Kaneko et al., 2021).
- *Demographic Representation (DR)* (Liang et al., 2022).
- *Stereotypical Association (SA)* (Liang et al., 2022).
- *HONEST* (Nozza et al., 2021).

3.2 Embedding Metrics

3.2.1 WEAT

The most famous embedding metric is given the Word Embedding Association Test (WEAT) (Caliskan et al., 2016), which aims to measure associations between demographic and neutral attributes. Demographic attributes are usually binary and denoted by A_1, A_2 , denoting two different societal groups (male and female, christians and atheist,...). Neutral attributes, on the other hand, are denoted by W_1, W_2 and represent two different stereotypes whose demographic association we are interested in.

$$s(a, W_1, W_2) = \sum_{w_1 \in \mathbb{W}_1} \frac{\cos(a, w_1)}{|\mathbb{W}_1|} - \sum_{w_2 \in \mathbb{W}_2} \frac{\cos(a, w_2)}{|\mathbb{W}_2|},$$

$$WEAT(A_1, A_2, W_1, W_2) = \frac{\sum_{a_1 \in A_1} s(a_1, W_1, W_2)/|A_1| - \sum_{a_2 \in A_2} s(a_2, W_1, W_2)/|A_2|}{\text{std}_{a \in A_1 \cup A_2} s(a, W_1, W_2)}.$$

class FairLangProc.metrics.embedding.**WEAT**(model: *Module*, tokenizer: *TokenizerType*, device: *str* = 'cuda')

Class for handling WEAT metric with a PyTorch model and tokenizer.

model

PyTorch model (e.g., BERT, GPT from HuggingFace).

Type

nn.Module

tokenizer

Tokenizer for the model.

Type

TokenizerType

device

Device to run the WEAT test on.

Type

str

metric(W1_words, W2_words, A1_words, A2_words, n_perm, pval)

Computation of the WEAT effect size between W1, W2 and A1, A2.

_get_embedding(outputs)

Abstract method whose implementation is required and which aims to compute the embedding of an output given by the model.

__init__(model: *Module*, tokenizer: *TokenizerType*, device: *str* = 'cuda') → *None*

Constructor for the WEAT class

Parameters

- **model** (*nn.Module*) – PyTorch model (e.g., BERT, GPT from HuggingFace).
- **tokenizer** (*TokenizerType*) – Tokenizer for the model.
- **device** (*str*) – Device to run the WEAT test on.

abstract _get_embedding(outputs)

Abstract method that instructs the class on how to obtain the embedding of a given input.

metric(W1_words: *list[str]*, W2_words: *list[str]*, A1_words: *list[str]*, A2_words: *list[str]*, n_perm: *int* = 10000, pval: *bool* = *True*) → *dict[str, float]*

Run WEAT test.

Parameters

- **W1_words** (*list[str]*) – Target concept 1 words/sentences
- **W2_words** (*list[str]*) – Target concept 2 words
- **A1_words** (*list[str]*) – Attribute 1 words/sentences
- **A2_words** (*list[str]*) – Attribute 2 words/sentences
- **n_perm** (*int*) – Number of permutations for p-value
- **pval** (*bool*) – Whether to compute or not the p-value

Returns

results – Dictionary with test results, namely mean similarity between W1, W2 and A1, A2; their sizes, the WEAT effect size and the p-value if needed.

Return type

`dict[str, float]`

3.3 Probability Metrics

3.3.1 LPBS

LPBS (Kurita et al., 2019) measures bias for a binary demographic group.

$$\text{LPBS} = \log \frac{p_1}{p_{\text{prior},1}} - \log \frac{p_2}{p_{\text{prior},2}}.$$

`FairLangProc.metrics.probability.LPBS(model: Module, tokenizer: TokenizerType, sentences: list[str], target_words: list[tuple[str]], fill_words: list[str], mask_indices: list[int] | None = None) → Tensor`

Computes LPBS score for a list of tuples of dimension 2 of target words.

Parameters

- **model** (*nn.Module*) – Language model used to compute probabilities.
- **tokenizer** (*TokenizerType*) – Tokenizer associated with the model.
- **sentences** (*list[str]*) – List of sentences with masks.
- **target_words** (*list[tuple[str]]*) – List containing tuples of words whose probabilities we want to compute.
- **fill_words** (*list[str]*) – List of words which replace the secondary mask.
- **mask_indices** (*list[int]*) – List of indices which indicate to which mask of the sentence each target word corresponds (i.e. first (0) or second (1)).

Returns

probs – List of LPBS scores

Return type

`torch.Tensor`

Example

```
>>> model = AutoModelForMaskedLM.from_pretrained('bert-base-uncased')
>>> tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
>>> sentences = ["[MASK] is a [MASK].", "[MASK] is a [MASK].", "The [MASK] was a_
↳ [MASK]."]
>>> target_words = [("John", "Mary"), ("He", "She"), ("man", "woman")]
>>> fill_words = ["engineer", "nurse", "doctor"]
>>> mask_indices = [0, 0, 1]
>>>
>>> LPBSScore = LPBS(
...     model = model,
...     tokenizer = tokenizer,
...     sentences = sentences,
...     target_words = target_words,
```

(continues on next page)

(continued from previous page)

```

...     fill_words = fill_words,
...     mask_indices = mask_indices
... )

```

3.3.2 CBS

CBS (Ahn et al., 2021) generalizes measurement of bias for non-binary demographic groups.

$$\text{CBS} = \text{Var}_{a \in \mathbb{A}} \log \frac{p_a}{p_{\text{prior},a}}.$$

FairLangProc.metrics.probability.CBS(model: Module, tokenizer: TokenizerType, sentences: list[str], target_words: list[tuple[str]], fill_words: list[str], mask_indices: list[int]) → Tensor

Computes CBS score for a list of tuples of dimension n of target words.

Parameters

- **model** (nn.Module) – Language model used to compute probabilities.
- **tokenizer** (TokenizerType) – Tokenizer associated with the model
- **sentences** (list[str]) – List of sentences with masks
- **target_words** (list[tuple[str]]) – List containing tuples of words whose probabilities we want to compute
- **fill_words** (list[str]) – List of words which replace the secondary mask
- **mask_indices** (list[int]) – List of indices which indicate to which mask of the sentence each target word corresponds (i.e. first (0) or second (1))

Returns

probs – List of CBS scores

Return type

torch.Tensor

Example

```

>>> model = AutoModelForMaskedLM.from_pretrained('bert-base-uncased')
>>> tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
>>> target_words = [("John", "Mamadouk", "Liu"), ("white", "black", "asian"), (
    ↪ "white", "black", "asian")]
>>> sentences = ["[MASK] is a [MASK]", "The [MASK] kid got [MASK] results", "The_
    ↪ [MASK] kid wanted to be a [MASK]"]
>>> fill_words = ["engineer", "outstanding", "doctor"]
>>> mask_indices = [0, 1, 1]
>>>
>>> CBSscore = CBS(
...     model = model,
...     tokenizer = tokenizer,
...     sentences = sentences,
...     target_words = target_words,
...     fill_words = fill_words,
...     mask_indices = mask_indices
... )

```


3.3.3 CPS

CPS (Nangia et al., 2020) uses sentence pairs which coincide in a series of unmodified tokens.

$$\text{CPS}(S) = \sum_{u \in U} \log \mathbb{P}(u | U \setminus u, A),$$

`FairLangProc.metrics.probability.CPS(model: Module, tokenizer: TokenizerType, sentences: list[str], target_words: list[str]) → list[float]`

Computes the CPS score for list of sentences.

Parameters

- **model** (*nn.Module*) – Language model used to compute probabilities.
- **tokenizer** (*TokenizerType*) – Tokenizer associated with the model.
- **sentences** (*list[str]*) – List of sentences for whom we will compute the CPS score.
- **target_words** (*list[str]*) – List of target words which should not be masked.

Returns

score – List of CPS score of the sentences.

Return type

list[float]

Example

```
>>> model = AutoModelForMaskedLM.from_pretrained("bert-base-uncased")
>>> tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
>>> sentences = ['The actor did a terrible job', 'The actress did a terrible job',
↪ 'The doctor was an exemplary man', 'The doctor was an exemplary woman']
>>> target_words = ['actor', 'actress', 'man', 'woman']
>>>
>>> CPSscore = CPS(
...     model = model,
...     tokenizer = tokenizer,
...     sentences = sentences,
...     target_words = target_words
... )
```

3.3.4 AUL

AUL (Kaneko et al., 2021) predicts the probability of all tokens in the sentence without masking.

$$\text{AUL}(S) = \frac{1}{|S|} \sum_{s \in S} \log \mathbb{P}(s | S).$$

`FairLangProc.metrics.probability.AUL(model: Module, tokenizer: TokenizerType, sentences: list[str]) → list[float]`

Computes the AUL score for list of sentences.

Parameters

- **model** (*nn.Module*) – Language model used to compute probabilities.
- **tokenizer** (*TokenizerType*) – Tokenizer associated with the model.
- **sentences** (*list[str]*) – List of sentences for whom we will compute the AUL score.

Returns

score – List of AUL score of the sentences.

Return type

`list[float]`

Example

```
>>> model = AutoModelForMaskedLM.from_pretrained("bert-base-uncased")
>>> tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
>>> sentences = ['The actor did a terrible job', 'The actress did a terrible job',
→ 'The doctor was an exemplary man', 'The doctor was an exemplary woman']
>>>
>>> AULscore = AUL(
...     model = model,
...     tokenizer = tokenizer,
...     sentences = sentences
... )
```

3.4 Generated Text Metrics

3.4.1 Demographic Representation

DR (Liang et al., 2022) is computed as follows:

$$\text{DR}(a) = \sum_{w_i \in \mathbb{A}} \sum_{\hat{Y} \in \hat{\mathbb{Y}}} C(w_i, \hat{Y}),$$

`FairLangProc.metrics.generated_text.DemRep(demWords: dict[str, list[str]], sentences: list[str]) → dict[str, int]`

Computes Demographic representation.

Parameters

- **demWords** (*dict[str, list[str]]*) – Dictionary whose keys represent demographic attributes and whose values represent words with demographic meaning.
- **sentences** (*list[str]*) – List of sentences to run the demographic representation.

Returns

demRepVect – Dictionary with demographic counts for all considered words and sentences.

Return type

`dict[str, int]`

Example

```
>>> gendered_words = {
...     'male': ['he', 'him', 'his'],
...     'female': ['she', 'her', 'actress', 'hers']
... }
>>> sentences = [
...     'She is such a good match to him.',
...     'He is trying way too hard to be an actor.',
...     'Her mother is trying to make ends meet.'
```

(continues on next page)

(continued from previous page)

```

...     'My aunt is baking, do you want to try?'
...     ]
>>>
>>> DR = DemRep(
...     sentences = sentences,
...     demWords = gendered_words
... )

```

3.4.2 Stereotypical Association

SA (Liang et al., 2022) is computed as follows:

$$DR(a) = \sum_{w_i \in \mathbb{A}} \sum_{\hat{Y} \in \hat{\mathbb{Y}}} C(w_i, \hat{Y}),$$

FairLangProc.metrics.generated_text.**StereoAsoc**(targetWords: *list[str]*, demWords: *dict[str, list[str]]*, sentences: *list[str]*) → *dict[str, dict[str, int]]*

Computes Stereotypical Association

Parameters

- **targetWords** (*list[str]*) – List of words whose associations we want to compute.
- **demWords** (*dict[str, list[str]]*) – Dictionary whose keys represent demographic attributes and whose values represent words with demographic meaning.
- **sentences** (*list[str]*) – List of sentences to run the stereotypical association.

Returns

steAsocVect – Dictionary which stores demographic counts for all considered words and sentences indexed by targetWords.

Return type

dict

Example

```

>>> gendered_words = {
...     'male': ['he', 'him', 'his'],
...     'female': ['she', 'her', 'actress', 'hers']
... }
>>> sentences = [
...     'She is such a good match to him.',
...     'He is trying way too hard to be an actor.',
...     'Her mother is trying to make ends meet.'
...     'My aunt is baking, do you want to try?'
... ]
>>> target_words = ['mother', 'baking']
>>>
>>> ST = StereoAsoc(
...     sentences = sentences,
...     demWords = gendered_words,
...     targetWords = target_words
... )

```

3.4.3 HONEST

HONEST (Nozza et al., 2021) measures how many of the top k completions contain harmful words.

$$\text{HONEST}(\hat{\mathbb{Y}}) = \frac{\sum_{\hat{Y}_k \in \hat{\mathbb{Y}}_k} \sum_{\hat{y} \in \hat{Y}_k} \mathbf{1}(\hat{y} \in \mathbb{Y}_{\text{hurt}})}{|\hat{\mathbb{Y}}|k}.$$

FairLangProc.metrics.generated_text.HONEST(completions: list[list[str]], hurtLex: list[str]) → float

Computes HONEST score for a set of completions given a hurt lexicon.

Parameters

- **completions** (list[list[str]]) – List of completions given by a model. This is a list of lists: it should contain lists of completions of the same size. The lists of completions are lists of constant size containing the completions of a given model for a certain prompt.
- **hurtLex** (list[str]) – List of words belonging to the hurt lexicon.

Returns

honestScore – HONEST score for the given sentences and hurt lexicon.

Return type

float

Example

```
>>> tokenizer = AutoTokenizer.from_pretrained('gpt2')
>>> model = AutoModelForCausalLM.from_pretrained('gpt2')
>>> corpus = [
...     'I cannot believe she did that, she is so',
...     'I cannot believe he did that, he is so',
...     'She tries and she tries to no avail. She is such a',
...     'He tries and she tries to no avail. He is such a',
... ]
>>> completions = []
>>> for prompt in corpus:
...     input_ids = tokenizer.encode(prompt, return_tensors="pt")
...     outputs = model.generate(
...         input_ids,
...         max_length=30,
...         num_return_sequences=3,
...         do_sample=True,
...         early_stopping=True
...     )
...     completions.append([tokenizer.decode(output, skip_special_tokens=True) for
↪output in outputs])
>>> hurtLex = ['fool', 'bitch', 'stupid', 'incompetent', 'idiot', 'dumb']
>>>
>>> honestScore = HONEST(
...     completions = completions,
...     hurtLex = hurtLex
... )
```

See also

- [Tutorials](#) - Interactive Jupyter notebooks ([DemoMetrics.ipynb](#)) for bias measurement

ALGORITHMS

FairLangProc provides a comprehensive suite of bias mitigation processors for Large Language Models.

4.1 Supported processors

Fairness processors can be classified according to their stage in the Machine Learning pipeline:

- **Pre-processors:** Fairness processors that modify the model inputs (data augmentation, embedding projection,...).
- **In-processors:** Fairness processors that modify the training process (regularizers, adapters,...).
- **Intra-processors:** Fairness processors that modify the model's behavior without further training (attention scaling,...).

The supported methods are:

- *Counterfactual Data Augmentation (CDA)* (Webster et al. 2020).
- *Projection based debiasing* (Bolukbasi et al., 2023).
- *Bias removal with No Demographics (BLIND)* (Orgad et al., 2023).
- *Adapter-based DEbiasing of Language models (ADELE)* (Lauscher et al., 2021).
- *Modular Debiasing with Diff Subnetworks* (Hauzenberger et al., 2023).
- *Entropy Attention Temperature (EAT) scaling* (Zayed et al., 2023).
- *Entropy Attention Regularizer (EAR)* (Attanasio et al., 2022).
- *Embedding based regularizer* (Liu et al., 2020).
- *Selective unfreezing* (Gira et al., 2024).

Note

Different algorithms have different trade-offs. See our tutorials for more detailed comparisons of fairness-accuracy tradeoffs.

4.1.1 Pre-processing

Pre-processors are fairness processors that modify the model inputs.

The supported methods are:

- *Counterfactual Data Augmentation (CDA)* (Webster et al. 2020).
- *Projection based debiasing* (Bolukbasi et al., 2023).

- *Bias removal with No Demographics (BLIND)* (Orgad et al., 2023).

Counterfactual Data Augmentation (CDA)

Data augmentation is the process of curating or upsampling the dataset to obtain a more representative distribution to train the model on. In particular, Counterfactual Data Augmentation (CDA) (Webster et al. 2020) consists of flipping words with demographic information while preserving semantic correctness. This procedure can be one-sided and discard the original sentence or two-sided to consider both the original and its augmented version.

`FairLangProc.algorithms.preprocessors.augmentation.CDA(batch: dict, pairs: dict[str, str], columns: list[str] | None = None, bidirectional: bool = True) → dict`

Perform CDA on a batch of training instances.

Parameters

- **batch** (`dict`) – Batch of training instances
- **pairs** (`dict`) – Dictionary of counterfactual pairs
- **columns** (`list[str]`) – List of columns on which CDA should be performed. If none, applies CDA to all columns.
- **bidirectional** (`bool`) – If true, applies bidirectional CDA (preserves original training instance). If false, deletes original training instance.

Returns

- **output** (`dict`) – Augmented training instance.
- **modified** (`dict`) – Whether or not the training instance was augmented.

Example

```
>>> from FairLangProc.algorithms.preprocessors import CDA
>>> gendered_pairs = [('he', 'she'), ('him', 'her'), ('his', 'hers'), ('actor',
↳ 'actress'), ('priest', 'nun'),
... ('father', 'mother'), ('dad', 'mom'), ('daddy', 'mommy'), ('waiter', 'waitress
↳ '), ('James', 'Jane')]
>>>
>>> cda_train = Dataset.from_dict(CDA(imdb['train'][:], pairs = dict(gendered_
↳ pairs)))
>>> train_CDA = cda_train.map(tokenize_function, batched=True)
>>> train_CDA.set_format(type="torch", columns=["input_ids", "attention_mask",
↳ "label"])
```

Projection-based debiasing

Projection-based debiasing methods (Bolukbasi et al., 2023) operate on latent space, looking to identify a bias subspace given by an orthogonal basis, $\{v_i\}_{i=1}^{n_{bias}}$. Then, the hidden representation of any input can be debiased by removing its projection onto this space, formally

$$h_{proj} = h - \sum_{i=1}^{n_{bias}} \langle h, v_i \rangle v_i.$$

This can be done either at the word or sentence level. In either case the bias subspace is generally identified through PCA, and usually its dimension is one, resulting in the construction of a bias direction.

BLIND debiasing

BLIND (Orgad et al., 2023) is a debiasing procedure based on a complementary classifier $g_B : \mathbb{R}^{d_L} \rightarrow \mathbb{R}$ with parameters θ_B , that takes the hidden representation vector as inputs and outputs the success probability of the model head for the downstream task. This probability is then used as a weight for said observation whose magnitude is controlled through a hyper-parameter $\gamma \geq 0$:

$$\mathcal{L}_{BLIND} = (1 - \sigma(g_B(h; \theta_B)))^\gamma \mathcal{L}^{task}(\hat{y}, y).$$

The term $\sigma(g_B(h; \theta_B))$ represents the model success probability for the downstream task: the bigger it is the less weight the observation has, while the smaller it is the more weight it carries. This forces the model to pay special attention to observations with low probability of success during training. Note that when $\gamma = 0$ the original loss function is restored, while $\gamma \gg 1$ exacerbates the effect of the reweighting.

4.1.2 In-processing

In-processors are fairness processors that modify the training process.

The supported methods are:

- *Adapter-based DEbiasing of Language models (ADELE)* (Lauscher et al., 2021).
- *Entropy Attention Regularizer (EAR)* (Attanasio et al., 2022).
- *Embedding based regularizer* (Liu et al., 2020).
- *Selective unfreezing* (Gira et al., 2024).

ADELE

The ADELE procedure (Lauscher et al., 2021) is based on the adapter framework. A single adapter module is included to each transformer layer after the feed-forward sub-layer, where the outputs are compressed to a bottleneck dimension m and then decompressed back to the hidden size of the transformer, d_L . The adapter module itself consists of a two-layer feed-forward network:

$$\text{Adapter}(\mathbf{h}, \mathbf{r}) = U \cdot g(D \cdot \mathbf{h}) + \mathbf{r}$$

where $\mathbf{h}$ and $\mathbf{r}$ are the hidden state and residual of the corresponding transformer layer, g is an activation function and $D \in \mathbb{R}^{m \times d_L}, U \in \mathbb{R}^{d_L \times m}$ represent the projection matrices. The idea behind the adapter layer is to introduce an information bottleneck which compresses the latent representation of the inputs, forcing the model to discard all irrelevant information.

class FairLangProc.algorithms.inprocessors.adapter.DebiasAdapter(*args: Any, **kwargs: Any)

Implements ADELE debiasing based on bottleneck adapter.

Example

```
>>> from adapters import AdapterTrainer
>>> from FairLangProc.algorithms.inprocessors import DebiasAdapter
>>>
>>> DebiasAdapter = DebiasAdapter(
...     model = AutoModel.from_pretrained('bert-base-uncased'),
...     adapter_config = "seq_bn"
... )
>>> AdeleModel = DebiasAdapter.get_model()
>>>
>>> trainer = AdapterTrainer(
```

(continues on next page)

(continued from previous page)

```

...     model=AdeleModel,
...     args=training_args,
...     train_dataset=train_CDA,
...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(AdeleModel.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> trainer.train()
>>> results = trainer.evaluate()
>>> print(results)

```

`__init__(model: torch.nn.Module, adapter_name: str = 'debias_adapter', adapter_config: str | dict = 'seq_bn') → None`

Constructor of the DebiasAdapter class.

Parameters

- **model** (*nn.Module*) – Pretrained model (e.g., BERT, GPT-2)
- **adapter_name** (*str*) – Tensor with ids of text with demographic information of group A
- **adapter_config** (*Union[str, dict]*) – Name or dictionary of the desired configuration for the adapter (bottleneck by default)

Embedding based regularizer

Embedding based regularizers (Liu et al., 2020) are based on the distance between the embeddings of counterfactual pairs given by $A\$,$

$$\mathcal{R} = \sum_{(a_i, a_j) \in A} \|M(a_i) - M(a_j)\|_2.$$

```

class FairLangProc.algorithms.inprocessors.regularizers.EmbeddingBasedRegularizer(*args:
    Any,
    **kwargs:
    Any)

```

Class for adding a regularizer based on the embeddings of counterfactual pairs. Requires the implementation of the `_get_embedding` method

Example

```

>>> from FairLangProc.algorithms.inprocessors import EmbeddingBasedRegularizer
>>> class BERTEmbeddingReg(EmbeddingBasedRegularizer):
...     def _get_embedding(self, inputs):
...         return self.model(**inputs).last_hidden_state[:,0,:]
>>> model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')
>>> tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
>>> words = [('he', 'she'), ('his', 'hers'), ('monk', 'nun')]
>>> EmbRegularizer = EARModel(
...     model = model,
...     tokenizer = tokenizer,
...     word_pairs = words,

```

(continues on next page)

(continued from previous page)

```

...     ear_reg_strength = 0.01
... )
>>>
>>> trainer = Trainer(
...     model=EARRegularizer,
...     args=training_args,
...     train_dataset=train_dataset,
...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(EARRegularizer.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> trainer.train()
>>> results = trainer.evaluate()
>>> print(results)

```

`__init__`(*model: torch.nn.Module, tokenizer: TokenizerType, word_pairs: list[tuple[str]], ear_reg_strength: float = 0.01*) → None

Constructor of the EmbeddingBasedRegularizer class.

Parameters

- **model** (*nn.Module*) – A language model
- **tokenizer** (*TokenizerType*) – Tokenizer of the model
- **word_pairs** (*list[tuple[str]]*) – List of tuples of counterfactual pairs whose embeddings should be close together (e.g. daughter and son, he and she,...).
- **ear_reg_strength** (*float*) – Hyper-parameter containing the strength of the regularization term.

EAR

EAR (Attanasio et al., 2022) tries to maximize the entropy of the attention weights to encourage attention to the broader context of the input,

$$\mathcal{R} = - \sum_{l=1}^L \text{entropy}_l(\mathbf{A})$$

where $\text{entropy}_l(\cdot)$ denotes the entropy of the l -th layer.

class FairLangProc.algorithms.inprocessors.regularizers.**EARModel**(*args: Any, **kwargs: Any)

Class for adding a regularizer based on entropy attention.

Example

```

>>> from FairLangProc.algorithms.inprocessors import EARModel
>>>
>>> model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')
>>> EARRegularizer = EARModel(
...     model = model,
...     ear_reg_strength = 0.01
... )

```

(continues on next page)

(continued from previous page)

```

>>>
>>> trainer = Trainer(
...     model=EARRegularizer,
...     args=training_args,
...     train_dataset=train_dataset,
...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(EARRegularizer.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> trainer.train()
>>> results = trainer.evaluate()
>>> print(results)

```

`__init__(model: torch.nn.Module, ear_reg_strength: float = 0.01)`

Constructor for the EARModel class

Parameters

- **model** (*nn.Module*) – A language model.
- **ear_reg_strength** (*float*) – Hyper-parameter containing the strength of the regularization term.

Selective unfreezing

Selective unfreezing (Gira et al., 2024) aims to circumvent catastrophic forgetting during fine-tuning by freezing a big amount of the model parameters, which also helps lessening computational expenses.

`FairLangProc.algorithms.inprocessors.selective Updating.selective_unfreezing(model: torch.nn.Module, substrings: list[str]) → None`

Freeze all model’s parameters and selectively unfreeze those specified in parameters.

Parameters

- **model** (*nn.Module*) – The model whose parameters will be adjusted.
- **substrings** (*list[str]*) – List of substrings to search for in parameter names.

Example

```

>>> from FairLangProc.algorithms.inprocessors import selective_unfreezing
>>>
>>> FrozenBert = AutoModelForSequenceClassification.from_pretrained('bert-base-
↳ uncased')
>>> selective_unfreezing(FrozenBert, ["attention.self", "attention.output"])
>>>
>>> trainer = Trainer(
...     model=FrozenBert,
...     args=training_args,
...     train_dataset=train_CDA,

```

(continues on next page)

(continued from previous page)

```

...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(FrozenBert.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> trainer.train()
>>> results = trainer.evaluate()
>>> print(results)

```

4.1.3 Intra-processing

Intra-processors are fairness processors that modify the model's behavior without further training.

The supported methods are:

- *Modular Debiasing with Diff Subnetworks* (Hauzenberger et al., 2023).
- *Entropy Attention Temperature (EAT) scaling* (Zayed et al., 2023).

MoDDiffy

MoDDiffy (Hauzenberger et al., 2023) creates many sparse subnetworks to address bias for different attributes (gender, religion,...) through the idea of *Diff* pruning. Basically, they freeze the model parameters, θ , and train another network with parameters, δ , with a loss function that promotes accuracy, sparsity and debiasing:

$$\mathcal{L}_\rho = \mathcal{L}_\rho^{task} + \lambda_\rho^0 \mathcal{L}_\rho^0 + \lambda_\rho^{debias} \mathcal{L}_\rho^{debias},$$

where:

- $\mathcal{L}_\rho^{task}$ represents the original loss of the downstream task with the new parameters.
- $\mathcal{L}_\rho^0$ promotes sparsity through a smooth approximation to the L_0 norm of the new parameters by means of the hard-concrete distribution of parameters $(\log \alpha_\rho, 1)$ and hyper-parameters $\gamma < 0, \zeta > 1$.
- $\mathcal{L}_\rho^{debias}$ debiases the outputs by approximating the mutual information of embeddings belonging to different demographic groups.

In particular the sparsity loss takes the form:

$$\mathcal{L}_\rho^0 = \sum_{i=1}^{|\delta_\rho|} \sigma \left(\log \alpha_{\rho,i} - \log \left(-\frac{\gamma}{\zeta} \right) \right).$$

And the debiasing loss:

$$\mathcal{L}_\rho^{debias} = \left(\frac{\sum_{x_A \in X_\rho^A} \phi(M(x_A))}{|X_\rho^A|} - \frac{\sum_{x_B \in X_\rho^B} \phi(M(x_B))}{|X_\rho^B|} \right)^2,$$

where ϕ is a transformation kernel.

```
class FairLangProc.algorithms.intraprocessors.modular.DiffPrunDebiasing(head: Module,
                                                                    encoder: Module,
                                                                    loss_fn: Callable,
                                                                    input_ids_A: Tensor,
                                                                    input_ids_B: Tensor,
                                                                    lambda_sparse: float
                                                                    = 1.0, lambda_bias:
                                                                    float = 1.0,
                                                                    bias_kernel: Callable
                                                                    | None = None,
                                                                    fixmask_init: bool =
                                                                    False, alpha_init: int |
                                                                    float | None = 5, struc-
                                                                    tured_diff_pruning:
                                                                    bool | None = False,
                                                                    upper: float = 1.1,
                                                                    lower: float = -0.1)
```

Implements differ pruning for bias mitigation in pretrained models.

Requires the implementation of the ‘_forward’ method, similar to ‘_get_embedding’ in other classes in that it should compute the embedding given some inputs.

Example

```
>>> from FairLangProc.algorithms.intraprocessors import DiffPrunBERT
>>>
>>> class DiffPrunBERT(DiffPrunDebiasing):
...     def _forward(self, input_ids, attention_mask=None, token_type_ids=None):
...         outputs = self.encoder(
...             input_ids = input_ids,
...             attention_mask = attention_mask,
...             token_type_ids = token_type_ids
...         )
...         return outputs.last_hidden_state[:,0,:]
>>>
>>> tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
>>> model = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased')
>>>
>>> gendered_pairs = [("manager", "manageress"), ("nephew", "niece"), ("prince",
    ↪ "princess"), ("baron", "baroness")]
>>> tokens_male = [words[0] for words in gendered_pairs]
>>> tokens_female = [words[1] for words in gendered_pairs]
>>> inputs_male = tokenizer(tokens_male, padding = True, return_tensors = "pt")
>>> inputs_female = tokenizer(tokens_female, padding = True, return_tensors = "pt")
>>>
>>> def normalize_by_column(x: torch.Tensor, eps: float = 1e-8):
...     mean = x.mean(dim=0, keepdim=True)
...     std = x.std(dim=0, keepdim=True)
...     return (x - mean) / (std + eps)
>>>
>>> ModularDebiasingBERT = DiffPrunBERT(
...     head = model.classifier,
...     encoder = model.bert,
```

(continues on next page)

(continued from previous page)

```

...     loss_fn = torch.nn.CrossEntropyLoss(),
...     input_ids_A = inputs_male,
...     input_ids_B = inputs_female,
...     bias_kernel = normalize_by_column,
...     upper = 10,
...     lower = -0.001,
...     lambda_bias = 0.5,
...     lambda_sparse = 0.00001
... )
>>> trainer = Trainer(
...     model=ModularDebiasingBERT,
...     args=training_args,
...     train_dataset=train_dataset,
...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(ModularDebiasingBERT.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> trainer.train()
>>> results = trainer.evaluate()
>>> print(results)

```

__init__(*head: Module, encoder: Module, loss_fn: Callable, input_ids_A: Tensor, input_ids_B: Tensor, lambda_sparse: float = 1.0, lambda_bias: float = 1.0, bias_kernel: Callable | None = None, fixmask_init: bool = False, alpha_init: int | float | None = 5, structured_diff_pruning: bool | None = False, upper: float = 1.1, lower: float = -0.1*)

Constructor of the DiffPrunDebiasing class.

Parameters

- **head** (*nn.Module*) – Head used for the task at hand (classification, question answering,...).
- **encoder** (*nn.Module*) – Pretrained model (e.g., BERT, GPT-2).
- **loss_fn** (*Callable*) – Loss function.
- **input_ids_A** (*torch.Tensor*) – Tensor with ids of text with demographic information of group A.
- **input_ids_B** (*torch.Tensor*) – Tensor with ids of text with demographic information of group B.
- **lambda_sparse** (*float*) – Weight for sparsity loss.
- **lambda_bias** (*float*) – Weight for bias mitigation loss.
- **bias_kernel** (*Callable*) – Kernel for the embeddings of the bias loss. If None, defaults to the identity.
- **fixmask_init** (*bool*) – If true, uses DiffWeightFixmask (i.e. only masks) instead of DiffWeightFinetune (i.e. smooth pruning).
- **alpha_init** (*Optional[Union[int, float]]*) – Initialization value for the log alpha parameters.
- **structured_diff_pruning** (*Optional[bool]*) – If true, adds a group structure to the diff pruning process (see DiffWeightFinetune)

- **upper** (*float*) – Parameter for concrete relaxation (has to be > 1).
- **lower** (*float*) – Parameter for concrete relaxation (has to be < 0).

EAT

EAT scaling (Zayed et al., 2023) modifies the distribution of the attention scores with a temperature-related parameter, $\beta \in [0, \infty)$:

$$\text{Attention}_{\beta}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\beta \mathbf{QK}}{\sqrt{d_k}}\right) \mathbf{V},$$

the idea being that when $\beta \gg 1$ the head attends only to the tokens with biggest scores while $\beta \approx 0$ forces the head to attend equally to all tokens. When $\beta = 1$ the attention head remains unmodified.

FairLangProc.algorithms.intraprocessors.redistribution.add_EAT_hook(model: *Module*, beta: *float* = 1.1)

Insert hook to modify attention scores.

Parameters

- **model** (*nn.Module*) – Model whose attention scores we want to modify.
- **beta** (*float*) – Temperature parameter.

Example

```
>>> from FairLangProc.algorithms.intraprocessors import add_EAT_hook
>>>
>>> EATBert = AutoModelForSequenceClassification.from_pretrained('bert-base-uncased
↪')
>>> beta = 1.5
>>> add_EAT_hook(model=EATBert, beta=beta)
>>>
>>> trainer = Trainer(
...     model=EATBert,
...     args=training_args,
...     train_dataset=train_dataset,
...     eval_dataset=val_dataset,
...     optimizers=(
...         AdamW(EATBert.parameters(), lr=1e-5, weight_decay=0.1),
...         None
...     )
... )
>>> results = trainer.evaluate()
>>> print(results)
```

See also

- *Tutorials* - Interactive Jupyter notebooks ([DemoProcessors notebook](#), [DemoDebiasing notebook](#)) for debiasing techniques

TUTORIALS

This section provides interactive tutorials demonstrating how to use FairLangProc for bias detection and mitigation in NLP models.

Tip

All tutorials are available as Jupyter notebooks. You can run them locally by cloning the repository or access them on Google Colab.

```
git clone https://github.com/arturo-perez-peralta/FairLangProc
```

Then navigate to the `notebooks/` directory and open the notebooks in Jupyter.

5.1 Datasets

Demo Datasets - [Open on GitHub](#)

Learn how to load and explore bias benchmark datasets.

- Load BBQ, StereoSet, BOLD, and other datasets
- Explore dataset structure and attributes

5.2 Metrics

Demo Metrics - [Open on GitHub](#)

Measure bias using various fairness metrics.

- WEAT for embedding-level bias
- LPBS, CBS, CPS, AUL for probability-level bias
- DR, ST, HONEST for generated text bias

5.3 Bias Mitigation

Demo Debiasing - [Open on GitHub](#)

Apply debiasing algorithms to reduce model bias.

- Pre-processing with CDA, BLIND and projection-based debiasing.
- In-processing with ADELE and regularizers.

- Intra-processing with MoDDiffy and EAT scaling.

5.4 End-to-End Workflow

Demo Processors - [Open on GitHub](#)

Use fairness processor to debias LLMs in the GLUE tasks.

- Use WEAT to measure bias.
- Use the different processors to mitigate embedding-level bias.
- Measure the impact on the performance metrics of the GLUE datasets.

5.5 Notebook Index

Notebook	Description
DemoDatasets.ipynb	Load and explore bias benchmark datasets
DemoMetrics.ipynb	Measure bias with WEAT, LPBS, CBS, HONEST, etc.
DemoDebiasing.ipynb	Apply debiasing algorithms (CDA, Projection, ADELE)
DemoProcessors.ipynb	Use in-processing and intra-processing bias mitigators

5.6 Requirements

To run the notebooks locally, you need to install FairLangProc and download the datasets:

```
pip install FairLangProc
```

Note

Some notebooks require significant computational resources (GPU recommended). For GPU acceleration, ensure CUDA is installed and configured.

```
# Verify CUDA availability
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
```


CITATION AND BIBLIOGRAPHY

6.1 Citation

If you use FairLangProc in your research, please cite our paper, **FairLangProc: A Python package for fairness in NLP**.

6.1.1 BibTeX

```
@misc{pérezperalta2025fairlangprocpythonpackagefairness,  
  title={FairLangProc: A Python package for fairness in NLP},  
  author={Arturo Pérez-Peralta and Sandra Benítez-Peña and Rosa E. Lillo},  
  year={2025},  
  eprint={2508.03677},  
  archivePrefix={arXiv},  
  primaryClass={cs.CL},  
  url={https://arxiv.org/abs/2508.03677},  
}
```

6.1.2 APA Style

Pérez-Peralta, A., Benítez-Peña, S., & Lillo, R. E. (2025). FairLangProc: A Python package for fairness in NLP. arXiv preprint arXiv:2508.03677.

6.1.3 IEEE Style

A. Pérez-Peralta, S. Benítez-Peña, and R. E. Lillo, “FairLangProc: A Python package for fairness in NLP,” arXiv preprint arXiv:2508.03677, 2025.

6.1.4 MLA Style

Pérez-Peralta, Arturo, et al. “FairLangProc: A Python package for fairness in NLP.” arXiv preprint arXiv:2508.03677 (2025).

6.2 Related Publications

If you use specific datasets, metrics, or algorithms from FairLangProc, please also cite the original papers:

6.2.1 Datasets

- **BBQ** (Bias Benchmark for Question Answering): Parrish, A., et al. (2021). BBQ: A hand-built bias benchmark for question answering. arXiv:2110.08193.
- **BEC-Pro** (Bias Evaluation Corpus for Professions): Bartl, M., et al. (2020). Uncovering Hidden Bias in NLP. arXiv:2010.14534.
- **BOLD** (Bias in Open-Ended Language Generation): Dhamala, J., et al. (2021). BOLD: A large-scale bias benchmark for language model evaluation. ACM FAccT.
- **BUG** (Bias evaluation Universal Benchmark): Levy, S., et al. (2021). A Unified Approach to Benchmarking and Mitigating Bias. arXiv:2109.03858.
- **CrowS-Pairs**: Nangia, N., et al. (2020). CrowS-Pairs: A Challenge Dataset for Measuring Social Biases in Masked Language Models. EMNLP.
- **GAP** (Gender Ambiguous Pronouns): Webster, K., et al. (2018). Mind the GAP: A systematic approach to measuring gender bias in coreference resolution. ACL.
- **HolisticBias**: Smith, E. M., et al. (2022). HolisticBias: Sensitivity analysis across sociodemographic identity. arXiv:2205.09209.
- **HONEST** (HOLingistic Neutrality Evalutation for gender-based Stereotypes): Nozza, D., et al. (2021). HONEST: Measuring Hurtful Language in Gendered Audio. NAACL.
- **StereoSet**: Nadeem, M., et al. (2020). StereoSet: Measuring stereotypical bias in language models. arXiv:2004.09456.
- **UnQover**: Li, T., et al. (2020). UnQovering Unintended Bias in Language Models through Compositional Reasoning. arXiv:2010.02428.
- **WinoBias+**: Vanmassenhove, E., et al. (2021). Toward Controllable Gender Bias Mitigation in Neural Machine Translation. arXiv:2109.06105.
- **WinoBias**: Zhao, J., et al. (2018). Gender Bias in Coreference Resolution. ACL.
- **WinoGender**: Rudinger, R., et al. (2018). Gender Bias in Neural Coreference Resolution. ACL.

6.2.2 Metrics

- **WEAT** (Word Embedding Association Test): Caliskan, A., Bryson, J. J., & Narayanan, A. (2017). Semantics derived automatically from language corpora contain human-like biases. Science.
- **LPBS** (Log Probability Bias Score): Kurita, K., et al. (2019). Measuring bias in contextualized word representations. ACL.
- **CBS** (Categorical Bias Score): Ahn, H., et al. (2021). Why Do Pretrained Language Models Cross-persona Conversational Tasks? EMNLP.
- **CPS** (CrowS-Pairs Score): Nangia, N., et al. (2020). CrowS-Pairs: A Challenge Dataset for Measuring Social Biases in Masked Language Models. EMNLP.
- **AUL** (All Unmasked Log-likelihood): Kaneko, M., et al. (2021). Gender and Racial Bias in Language Models. arXiv:2104.07496.
- **DR** (Demographic Representation): Liang, P. P., et al. (2022). Towards Fairness in Text Generation. arXiv:2211.09110.
- **SA** (Stereotypical Association): Liang, P. P., et al. (2022). Towards Fairness in Text Generation. arXiv:2211.09110.
- **HONEST** (HOLingistic Neutrality Evalutation for gender-based Stereotypes): Nozza, D., Bianchi, F., & Hovy, D. (2021). HONEST: Measuring Hurtful Language in Gendered Audio. NAACL.

6.2.3 Algorithms

Pre-processing:

- **CDA** (Counterfactual Data Augmentation): Webster, K., et al. (2020). Measuring and Reducing the Correlation Between Word Embeddings and Gendered Constructs. ACL.
- **Projection-based Debiasing**: Bolukbasi, M., et al. (2016). Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings. NeurIPS.
- **BLIND** (Bias removalL wIth No Demographics): Orgad, H., et al. (2023). BLIND: Bias Removal via Debiasing at Inference Time. ACL.

In-processing:

- **ADELE** (Adapter-based DEbiasing of Language models): Lauscher, A., et al. (2021). AdapterFusion: Non-invasive Transfer of Knowledge for Text Classification. arXiv:2109.03646.
- **EAR** (Entropy Attention Regularizer): Attanasio, G., et al. (2022). SENTeX: BERT-based Sentiment Analysis for Italian. arXiv:2203.09192.
- **Embedding-based Regularizer**: Liu, Y., et al. (2020). Fairness without Demographics through Adversarial Learning. arXiv:1910.10486.
- **Selective Unfreezing**: Gira, M., et al. (2022). Selective Knowledge Transfer for Democratizing NLP. ACL.

Intra-processing:

- **MoDDiffy** (Modular Debiasing with Diff Subnetworks): Hauenberger, L., et al. (2023). Modular Debiasing for Language Models. ACL Findings.
- **EAT** (Entropy Attention Temperature scaling): Zayed, A., et al. (2023). DeBiasing: Learning about Bias Mitigation from Plain Models. arXiv:2305.13088.